

Universidade de Brasília
Arquitetura e Organização de Computadores

Projeto e Simulação de uma ULA RISC-V de 32 bits em VHDL

Autor: Victor Enéias Oliveira
Curso de Engenharia de Computação
Brasília — 2026

Sumário

1. Descrição do trabalho	3
2. Especificação e arquitetura da ULA	3
2.1. Interface	3
2.2. Conjunto de operações	3
2.3. Estrutura da arquitetura	4
3. Comparações com sinal e sem sinal	5
4. Detecção de overflow em ADD e SUB	5
4.1. Overflow na adição (ADD)	5
4.2. Overflow na subtração (SUB)	6
4.3. Método alternativo: vai-um interno	6
5. Simulação e verificação	7
5.1. Formas de onda	7
6. Conclusão	10
7. Código-fonte	11
7.1. ULA (ulaRV.vhd)	11
7.2. Testbench (tb_ulaRV.vhd)	14
8. Referências	17

1. Descrição do trabalho

Este trabalho apresenta o projeto, a implementação e a verificação de uma Unidade Lógica e Aritmética (ULA) de 32 bits inspirada na arquitetura RISC-V, descrita em VHDL. A ULA é o bloco combinacional responsável por executar as operações aritméticas, lógicas, de deslocamento e de comparação de um processador. No RISC-V, ela é acionada, por exemplo, nas instruções do tipo R (como `add`, `sub`, `and`, `slt`) e do tipo I, sendo a operação selecionada por um código de controle.

A ULA implementada possui duas entradas de dados de 32 bits (`A` e `B`), uma saída de dados de 32 bits (`Z`), uma entrada de seleção de 4 bits (`opcode`) e uma saída de condição de 1 bit (`cond`). O sinal `cond` indica se a condição testada por uma operação de comparação é verdadeira (1) ou falsa (0); para as operações que não são de comparação, ele permanece inativo em 0. Ao todo são suportadas quatorze operações, conforme a tabela da seção seguinte.

A descrição foi feita em nível comportamental, empregando a biblioteca `numeric_std` para as operações aritméticas e de comparação sobre os vetores lógicos. A lógica foi organizada dentro de um único processo combinacional, utilizando o comando `case-when` para selecionar a operação a partir do `opcode`. A verificação foi realizada por meio de um *testbench* autoverificável (self-checking), que aplica estímulos a cada operação e compara automaticamente a saída obtida com o valor esperado.

2. Especificação e arquitetura da ULA

2.1. Interface

A entidade `ulaRV` declara um *generic* `WSIZE` (largura da palavra, com valor padrão 32), o que permite reaproveitar a mesma descrição para outras larguras sem alterar o corpo da arquitetura. A interface é a seguinte:

```
entity ulaRV is
  generic (WSIZE : natural := 32);
  port (
    opcode : in  std_logic_vector(3 downto 0);
    A, B   : in  std_logic_vector(WSIZE-1 downto 0);
    Z      : out std_logic_vector(WSIZE-1 downto 0);
    cond   : out std_logic
  );
end entity ulaRV;
```

Listagem 1 — Interface (entity) da ULA.

2.2. Conjunto de operações

A tabela a seguir resume as quatorze operações suportadas e seus respectivos códigos de seleção (`opcode`):

Operação	Significado	OpCode
ADD	$Z \leftarrow A + B$ (soma com sinal)	0000
SUB	$Z \leftarrow A - B$ (subtração com sinal)	0001
AND	$Z \leftarrow A \text{ and } B$ (bit a bit)	0010
OR	$Z \leftarrow A \text{ or } B$ (bit a bit)	0011
XOR	$Z \leftarrow A \text{ xor } B$ (bit a bit)	0100
SLL	$Z \leftarrow A$ deslocado B bits à esquerda	0101
SRL	$Z \leftarrow A$ deslocado B bits à direita, lógico	0110
SRA	$Z \leftarrow A$ deslocado B bits à direita, aritmético	0111
SLT	$Z = 1$ se $A < B$, com sinal	1000
SLTU	$Z = 1$ se $A < B$, sem sinal	1001
SGE	$Z = 1$ se $A \geq B$, com sinal	1010
SGEU	$Z = 1$ se $A \geq B$, sem sinal	1011
SEQ	$Z = 1$ se $A = B$	1100
SNE	$Z = 1$ se $A \neq B$	1101

2.3. Estrutura da arquitetura

O corpo da arquitetura `behavioral` é composto por um único processo sensível a `opcode`, `A` e `B`. Como se trata de lógica puramente combinacional, a lista de sensibilidade contém todas as entradas lidas, garantindo que a saída seja recalculada sempre que qualquer estímulo mudar. Algumas decisões de projeto merecem destaque:

- **Atribuições padrão para evitar latch.** No início do processo, `z` recebe zero e `cond` recebe 0. Isso garante que todo caminho do `case` atribua um valor conhecido às saídas, impedindo a inferência de latches — um erro clássico em descrições combinacionais incompletas.
- **Uso de `numeric_std`.** As entradas são convertidas para `signed` ou `unsigned` conforme a semântica da operação. A soma e a subtração usam `signed`, pois em complemento de dois o mesmo somador serve tanto para operandos com sinal quanto sem sinal — apenas a interpretação do resultado muda.
- **Quantidade de deslocamento (`shamt`).** Assim como no RISC-V de 32 bits, apenas os 5 bits menos significativos de `B` são usados como quantidade de deslocamento, extraídos por `to_integer(unsigned(B(4 downto 0)))`. Isso limita o deslocamento à faixa de 0 a 31 bits, coerente com a largura da palavra.
- **Deslocamentos lógico e aritmético.** O `SRL` trata `A` como `unsigned` (preenche com zeros), enquanto o `SRA` trata `A` como `signed` (replica o bit de sinal), preservando o valor com sinal ao dividir por potências de dois.

- **Sinal cond nas comparações.** Nas operações de comparação (`SLT`, `SLTU`, `SGE`, `SGEU`, `SEQ`, `SNE`), a saída `z` recebe 1 quando a condição é verdadeira e 0 caso contrário, e o sinal `cond` acompanha esse resultado. Nas demais operações, `cond` permanece em 0.

3. Comparações com sinal e sem sinal

A diferença fundamental está na *interpretação* que se dá ao mesmo padrão de bits, e não nos bits em si. Um vetor de 32 bits é apenas uma sequência de zeros e uns; o que decide se ele representa um número positivo grande ou um número negativo é a convenção adotada na comparação.

Na interpretação **sem sinal** (*unsigned*), todos os 32 bits contribuem para a magnitude, e os valores vão de 0 a 4.294.967.295. Na interpretação **com sinal** (*signed*), usa-se a representação em complemento de dois: o bit mais significativo (MSB) passa a ter peso negativo, de modo que os valores vão de $-2.147.483.648$ a $+2.147.483.647$. Um MSB igual a 1 significa “número negativo” no caso com sinal, mas apenas “bit de maior peso ligado” no caso sem sinal.

O exemplo mais ilustrativo é o padrão `0xFFFFFFFF`. Interpretado como *unsigned*, ele vale 4.294.967.295 (o maior valor possível). Interpretado como *signed*, ele vale -1 (o menor em módulo entre os negativos). Por isso, ao comparar `0xFFFFFFFF` com `0x00000001`:

- `SLT` (com sinal): $-1 < 1$ é **verdadeiro** $\rightarrow Z = 1$.
- `SLTU` (sem sinal): $4.294.967.295 < 1$ é **falso** $\rightarrow Z = 0$.

Ou seja, o mesmo par de operandos produz resultados opostos apenas por causa da convenção de sinal. Em VHDL, essa escolha é feita explicitamente ao converter os vetores: `signed(A) < signed(B)` realiza a comparação com sinal, enquanto `unsigned(A) < unsigned(B)` realiza a comparação sem sinal. Os operadores da biblioteca `numeric_std` já implementam corretamente cada semântica; ao programador cabe apenas escolher o tipo adequado.

Do ponto de vista de hardware, ambas as comparações podem ser derivadas da subtração $A - B$. Na comparação *sem sinal*, $A < B$ equivale à ocorrência de um “empréstimo” (*borrow*) na subtração — isto é, o *vai-um* de saída indica qual operando é maior. Já na comparação *com sinal*, $A < B$ é dado pela expressão $N \oplus V$, onde N é o bit de sinal do resultado e V é o indicador de overflow; o overflow precisa entrar na conta justamente porque, quando a subtração estoura a faixa representável, o bit de sinal sozinho passa a mentir sobre o sinal verdadeiro do resultado. É por essa razão que o RISC-V oferece instruções separadas (`slt` e `sltu`): a distinção não é de conveniência, mas de lógica de hardware distinta.

4. Detecção de overflow em ADD e SUB

A ULA aqui descrita não sinaliza overflow — o RISC-V base (RV32I) também não gera exceção de overflow aritmético, delegando essa verificação ao software. Ainda assim, o overflow com

sinal pode ser detectado com lógica combinacional simples a partir dos bits de sinal dos operandos e do resultado.

Overflow (com sinal) ocorre quando o resultado verdadeiro não cabe na faixa representável de 32 bits em complemento de dois. A observação-chave é sobre os *sinais*:

4.1. Overflow na adição (ADD)

Na soma $A + B$, o overflow só pode acontecer quando os dois operandos têm o **mesmo sinal**. Somar um positivo com um negativo nunca estoura, pois o resultado fica entre eles. Quando A e B têm o mesmo sinal, mas o resultado Z aparece com sinal **diferente**, houve overflow. Em termos dos bits de sinal (bit 31):

```
ovf_add = (A(31) = B(31)) and (Z(31) /= A(31));

-- equivalente, em forma de soma de produtos:
ovf_add = ( A(31) and B(31) and not Z(31) )
          or ( not A(31) and not B(31) and Z(31) );
```

Listagem 2 — Detecção de overflow na adição com sinal.

Interpretando: “dois positivos que resultam em negativo” (linha superior) ou “dois negativos que resultam em positivo” (linha inferior).

4.2. Overflow na subtração (SUB)

Na subtração $A - B$, o overflow só pode ocorrer quando os operandos têm **sinais diferentes** (subtrair um negativo é somar um positivo, e vice-versa). Nesse caso, há overflow quando o sinal do resultado difere do sinal de A :

```
ovf_sub = (A(31) /= B(31)) and (Z(31) /= A(31));

-- equivalente, em forma de soma de produtos:
ovf_sub = ( not A(31) and B(31) and Z(31) )
          or ( A(31) and not B(31) and not Z(31) );
```

Listagem 3 — Detecção de overflow na subtração com sinal.

4.3. Método alternativo: vai-um interno

Uma formulação equivalente e muito usada em hardware é comparar o vai-um que *entra* no bit de sinal com o vai-um que *sai* dele: há overflow quando eles diferem, ou seja, $V = C_{in} \oplus C_{out}$. Em VHDL, uma forma prática de obter esses dois vai-uns é estender os operandos em um bit e observar os dois bits mais altos do resultado:

```
-- Estende A e B para WSIZE+1 bits antes de somar.
signal a_ext, b_ext, s_ext : signed(WSIZE downto 0);

a_ext <= resize(signed(A), WSIZE+1);
b_ext <= resize(signed(B), WSIZE+1);
s_ext <= a_ext + b_ext;                -- para SUB: a_ext - b_ext
```

```
-- Overflow = os dois bits de sinal do resultado estendido diferem.
ovf <= s_ext(WSIZE) xor s_ext(WSIZE-1);
```

Listagem 4 — Detecção de overflow via bit de sinal estendido.

Observação sobre o caso sem sinal: para operações *unsigned*, o que interessa não é o overflow de sinal, e sim o vai-um (carry) de saída na adição ou o empréstimo (borrow) na subtração. Esse indicador é obtido diretamente do bit `s_ext(WSIZE)` após somar/subtrair os operandos estendidos em um bit tratados como *unsigned*.

5. Simulação e verificação

A verificação foi feita com um *testbench* autoverificável. Em vez de conferir as formas de onda apenas visualmente, o *testbench* contém um procedimento `check` que aplica um estímulo (opcode, A e B), aguarda um período de estabilização e usa asserções (`assert`) para comparar automaticamente a saída `z` e o sinal `cond` com os valores esperados. Quando algum resultado diverge, uma mensagem de erro é emitida identificando o teste; quando tudo confere, registra-se uma nota “OK”. Isso torna a regressão rápida e confiável, pois erros são apontados diretamente pelo simulador.

Conforme solicitado, há ao menos um teste para cada operação da ULA, e as operações aritméticas (ADD e SUB) são exercitadas para resultado **positivo, negativo e zero**. Os casos de comparação cobrem tanto a condição verdadeira quanto a falsa, e as comparações com e sem sinal são testadas com o padrão `0xFFFFFFFF` justamente para evidenciar a diferença de interpretação discutida na seção 3.

Resumo dos casos de teste aplicados:

- ADD: $10 + 5 = 15$ (positivo); $-10 + 4 = -6$ (negativo); $-8 + 8 = 0$ (zero).
- SUB: $20 - 5 = 15$ (positivo); $5 - 20 = -15$ (negativo); $7 - 7 = 0$ (zero).
- Lógicas: AND, OR e XOR sobre padrões de bits conhecidos.
- Deslocamentos: SLL ($1 < 4$), SRL e SRA sobre `0x80000000`, evidenciando a diferença entre deslocamento lógico e aritmético.
- Comparações com sinal: SLT e SGE, casos verdadeiro e falso.
- Comparações sem sinal: SLTU e SGEU, casos verdadeiro e falso.
- Igualdade/diferença: SEQ e SNE, casos verdadeiro e falso.

5.1. Formas de onda

As figuras a seguir apresentam as formas de onda obtidas na simulação (realizada com o simulador GHDL, com o padrão VHDL-2008; o mesmo testbench é compatível com o ModelSim-Altera e o EDA Playground), evidenciando o comportamento dos sinais `opcode`, `A`, `B`,

z e cond ao longo do tempo. Cada operação ocupa uma janela de 10 ns. Nas figuras de operações aritméticas os operandos e o resultado são apresentados em decimal com sinal, e nas demais em hexadecimal, para melhor leitura.

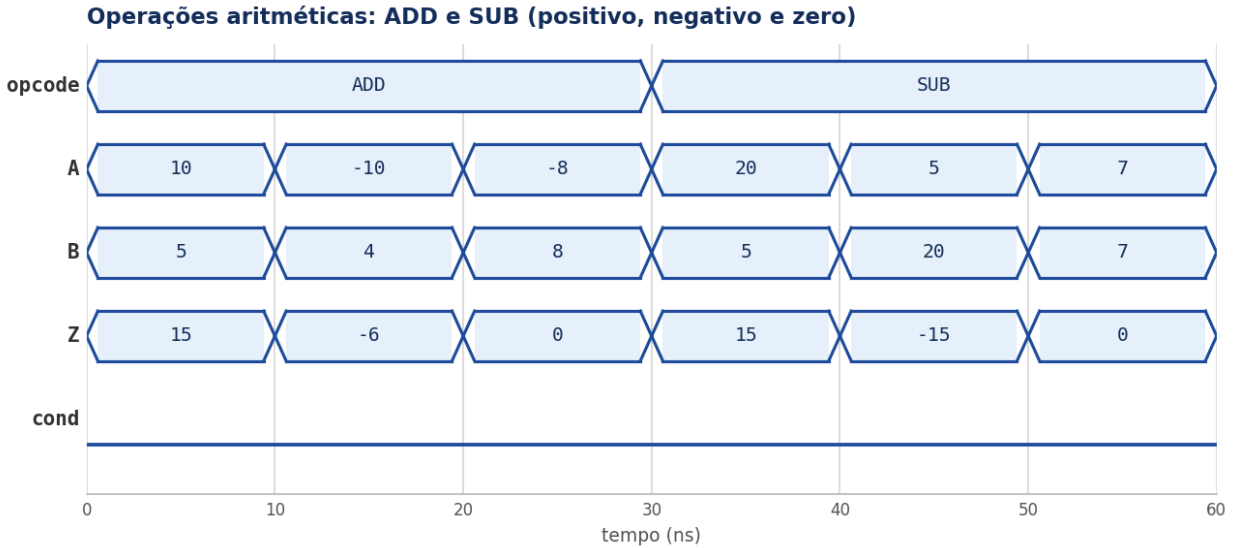


Figura 1 — ADD e SUB para resultados positivo, negativo e zero. O sinal cond permanece em 0 (não são operações de comparação).

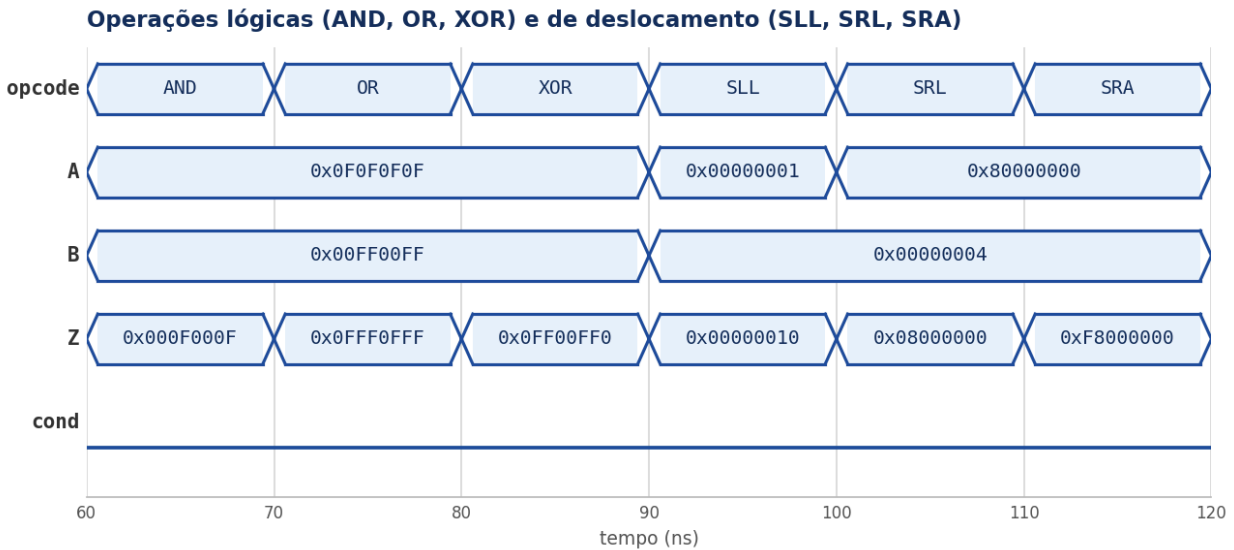


Figura 2 — Operações lógicas (AND, OR, XOR) e de deslocamento. Note SRL de 0x80000000 = 0x08000000 (lógico, preenche com zero) contra SRA = 0xF8000000 (aritmético, replica o sinal).

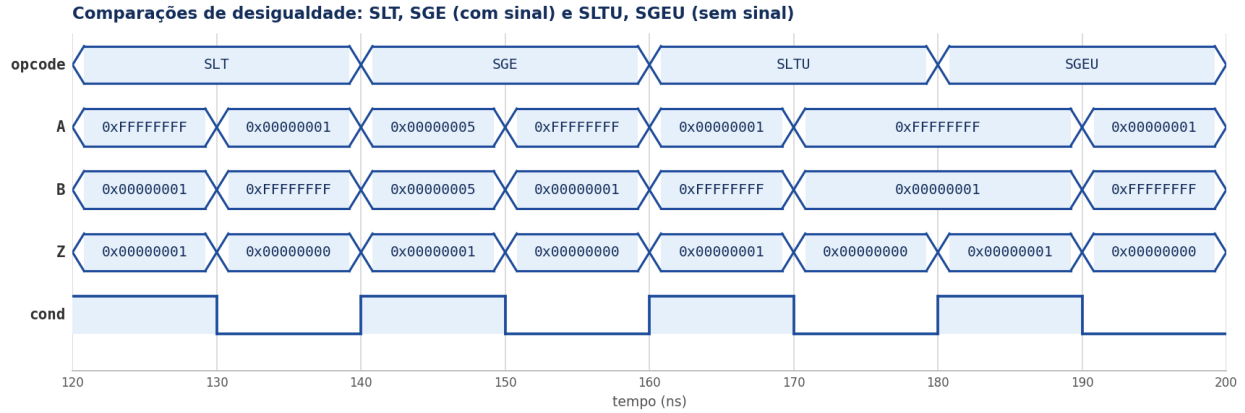


Figura 3 — Comparações de desigualdade. Evidencia a diferença de interpretação: *SLT* ($-1 < 1$) resulta verdadeiro (*cond* = 1), enquanto *SLTU* ($0xFFFFFFFF < 1$) resulta falso (*cond* = 0).

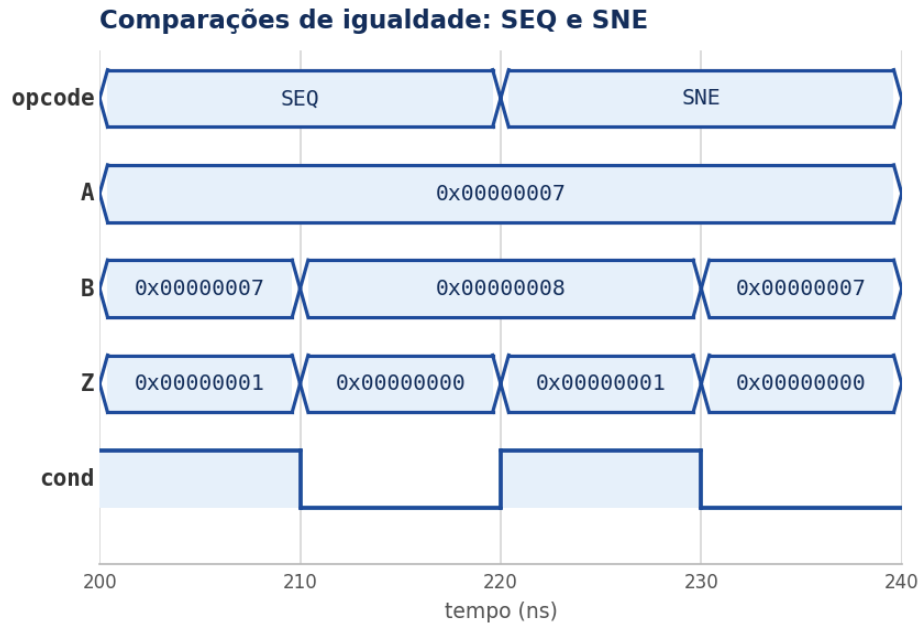


Figura 4 — Comparações de igualdade *SEQ* e *SNE*, com o sinal *cond* acompanhando o resultado de cada teste.

Por fim, o *transcript* do simulador confirma que todas as operações produziram exatamente os valores esperados. Cada asserção do *testbench* emitiu a nota “OK” correspondente, e nenhum erro foi reportado:

```

# GHDL 4.1.0 - simulação de ula_tb (std=08)
# vsim/run -all

OK    ADD positivo (10 + 5 = 15)
OK    ADD negativo (-10 + 4 = -6)
OK    ADD zero (-8 + 8 = 0)
OK    SUB positivo (20 - 5 = 15)
OK    SUB negativo (5 - 20 = -15)
OK    SUB zero (7 - 7 = 0)
OK    AND (0F0F & 00FF = 000F)
OK    OR (0F0F | 00FF = 0FFF)
OK    XOR (0F0F ^ 00FF = 0FF0)
OK    SLL (1 << 4 = 16)
OK    SRL (0x80000000 >> 4 sem sinal = 0x08000000)
OK    SRA (0x80000000 >> 4 com sinal = 0xF8000000)
OK    SLT verdadeiro (-1 < 1)
OK    SLT falso (1 < -1)
OK    SGE verdadeiro (5 >= 5)
OK    SGE falso (-1 >= 1)
OK    SLTU verdadeiro (1 < 0xFFFFFFFF)
OK    SLTU falso (0xFFFFFFFF < 1)
OK    SGEU verdadeiro (0xFFFFFFFF >= 1)
OK    SGEU falso (1 >= 0xFFFFFFFF)
OK    SEQ verdadeiro (7 == 7)
OK    SEQ falso (7 == 8)
OK    SNE verdadeiro (7 != 8)
OK    SNE falso (7 != 7)

>> Todos os testes da ULA foram executados.
>> 0 erros de asserção.

```

Figura 5 — Transcript da simulação: as 24 asserções passaram (“OK”) e a mensagem final confirma a execução completa, sem erros.

6. Conclusão

A ULA RISC-V de 32 bits foi projetada, descrita em VHDL e verificada com sucesso. A descrição comportamental baseada em `case-when` e na biblioteca `numeric_std` mostrou-se clara e diretamente mapeável para o conjunto de operações especificado, incluindo o tratamento correto de deslocamentos lógicos e aritméticos e a geração do sinal de condição. O *testbench* autoverificável cobriu todas as operações e, para as aritméticas, os três casos de resultado (positivo, negativo e zero), confirmando a corretude da implementação sem depender de inspeção manual das ondas.

As duas questões conceituais foram tratadas: a distinção entre comparações com e sem sinal, que decorre da interpretação do mesmo padrão de bits em complemento de dois, e a detecção de overflow em ADD e SUB, que pode ser obtida por lógica simples sobre os bits de sinal dos operandos e do resultado, ou equivalentemente pela comparação entre os vai-uns de entrada e saída do bit mais significativo.

7. Código-fonte

7.1. ULA (ulaRV.vhd)

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- ULA do RISC-V de 32 bits.
--
-- Entradas de dados: A e B (WSIZE bits).
-- Saida de dados:      Z      (WSIZE bits).
-- Selecao:             opcode (4 bits) - escolhe a operacao.
-- Saida de condicao: cond  - vale '1' quando a comparacao testada e
--                          verdadeira e '0' caso contrario. Para as
--                          operacoes que nao sao de comparacao cond fica
--                          inativo ('0').
entity ulaRV is
  generic (WSIZE : natural := 32);
  port (
    opcode : in  std_logic_vector(3 downto 0);
    A, B   : in  std_logic_vector(WSIZE-1 downto 0);
    Z      : out std_logic_vector(WSIZE-1 downto 0);
    cond   : out std_logic
  );
end entity ulaRV;

architecture behavioral of ulaRV is

  -- Codigos das operacoes (opcode).
  constant ADD_OP  : std_logic_vector(3 downto 0) := "0000";
  constant SUB_OP  : std_logic_vector(3 downto 0) := "0001";
  constant AND_OP  : std_logic_vector(3 downto 0) := "0010";
  constant OR_OP   : std_logic_vector(3 downto 0) := "0011";
  constant XOR_OP  : std_logic_vector(3 downto 0) := "0100";
  constant SLL_OP  : std_logic_vector(3 downto 0) := "0101";
  constant SRL_OP  : std_logic_vector(3 downto 0) := "0110";
  constant SRA_OP  : std_logic_vector(3 downto 0) := "0111";
  constant SLT_OP  : std_logic_vector(3 downto 0) := "1000";
  constant SLTU_OP : std_logic_vector(3 downto 0) := "1001";
  constant SGE_OP  : std_logic_vector(3 downto 0) := "1010";
  constant SGEU_OP : std_logic_vector(3 downto 0) := "1011";
  constant SEQ_OP  : std_logic_vector(3 downto 0) := "1100";
  constant SNE_OP  : std_logic_vector(3 downto 0) := "1101";

begin

  process (opcode, A, B)
    -- Quantidade de bits de deslocamento: apenas os bits menos
    -- significativos de B sao usados (5 bits para 32 bits).
    variable shamt : natural;
  begin
    -- Valores padrao: evitam a inferencia de latch.
    Z   <= (others => '0');
    cond <= '0';

    shamt := to_integer(unsigned(B(4 downto 0)));

    case opcode is

      when ADD_OP =>

```

```

        Z <= std_logic_vector(signed(A) + signed(B));

when SUB_OP =>
    Z <= std_logic_vector(signed(A) - signed(B));

when AND_OP =>
    Z <= A and B;

when OR_OP =>
    Z <= A or B;

when XOR_OP =>
    Z <= A xor B;

when SLL_OP =>
    Z <= std_logic_vector(shift_left(unsigned(A), shamt));

when SRL_OP =>
    Z <= std_logic_vector(shift_right(unsigned(A), shamt));

when SRA_OP =>
    Z <= std_logic_vector(shift_right(signed(A), shamt));

-- Set less than (com sinal): Z = 1 se A < B.
when SLT_OP =>
    if signed(A) < signed(B) then
        Z <= std_logic_vector(to_unsigned(1, WSIZE));
        cond <= '1';
    else
        Z <= (others => '0');
        cond <= '0';
    end if;

-- Set less than (sem sinal): Z = 1 se A < B.
when SLTU_OP =>
    if unsigned(A) < unsigned(B) then
        Z <= std_logic_vector(to_unsigned(1, WSIZE));
        cond <= '1';
    else
        Z <= (others => '0');
        cond <= '0';
    end if;

-- Set greater or equal (com sinal): Z = 1 se A >= B.
when SGE_OP =>
    if signed(A) >= signed(B) then
        Z <= std_logic_vector(to_unsigned(1, WSIZE));
        cond <= '1';
    else
        Z <= (others => '0');
        cond <= '0';
    end if;

-- Set greater or equal (sem sinal): Z = 1 se A >= B.
when SGEU_OP =>
    if unsigned(A) >= unsigned(B) then
        Z <= std_logic_vector(to_unsigned(1, WSIZE));
        cond <= '1';
    else
        Z <= (others => '0');
        cond <= '0';
    end if;

```

```
-- Set equal: Z = 1 se A == B.
when SEQ_OP =>
  if A = B then
    Z    <= std_logic_vector(to_unsigned(1, WSIZE));
    cond <= '1';
  else
    Z    <= (others => '0');
    cond <= '0';
  end if;

-- Set not equal: Z = 1 se A != B.
when SNE_OP =>
  if A /= B then
    Z    <= std_logic_vector(to_unsigned(1, WSIZE));
    cond <= '1';
  else
    Z    <= (others => '0');
    cond <= '0';
  end if;

when others =>
  Z    <= (others => '0');
  cond <= '0';

end case;
end process;

end architecture behavioral;
```

Listagem 5 — Descrição VHDL da ULA.

7.2. Testbench (tb_ulaRV.vhd)

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ula_tb is
end entity ula_tb;

architecture tb_arch of ula_tb is

    constant WSIZE : natural := 32;

    -- Sinais de estímulo e observação.
    signal opcode : std_logic_vector(3 downto 0);
    signal A, B   : std_logic_vector(WSIZE-1 downto 0);
    signal Z      : std_logic_vector(WSIZE-1 downto 0);
    signal cond   : std_logic;

    -- Declaração do componente ULA.
    component ulaRV is
        generic (WSIZE : natural := 32);
        port (
            opcode : in  std_logic_vector(3 downto 0);
            A, B   : in  std_logic_vector(WSIZE-1 downto 0);
            Z      : out std_logic_vector(WSIZE-1 downto 0);
            cond   : out std_logic
        );
    end component;

    -- Códigos das operações.
    constant ADD_OP : std_logic_vector(3 downto 0) := "0000";
    constant SUB_OP : std_logic_vector(3 downto 0) := "0001";
    constant AND_OP : std_logic_vector(3 downto 0) := "0010";
    constant OR_OP  : std_logic_vector(3 downto 0) := "0011";
    constant XOR_OP : std_logic_vector(3 downto 0) := "0100";
    constant SLL_OP : std_logic_vector(3 downto 0) := "0101";
    constant SRL_OP : std_logic_vector(3 downto 0) := "0110";
    constant SRA_OP : std_logic_vector(3 downto 0) := "0111";
    constant SLT_OP : std_logic_vector(3 downto 0) := "1000";
    constant SLTU_OP : std_logic_vector(3 downto 0) := "1001";
    constant SGE_OP : std_logic_vector(3 downto 0) := "1010";
    constant SGEU_OP : std_logic_vector(3 downto 0) := "1011";
    constant SEQ_OP : std_logic_vector(3 downto 0) := "1100";
    constant SNE_OP : std_logic_vector(3 downto 0) := "1101";

begin

    -- Instanciamento da ULA (dispositivo sob teste).
    dut: ulaRV
        generic map (WSIZE => WSIZE)
        port map (
            opcode => opcode,
            A      => A,
            B      => B,
            Z      => Z,
            cond   => cond
        );

    stimulus: process

        constant PERIODO : time := 10 ns;
    
```

```

-- Aplica um estímulo e confere o resultado esperado em Z e cond.
procedure check(
    constant nome      : in string;
    constant op        : in std_logic_vector(3 downto 0);
    constant a_in      : in std_logic_vector(WSIZE-1 downto 0);
    constant b_in      : in std_logic_vector(WSIZE-1 downto 0);
    constant z_esp      : in std_logic_vector(WSIZE-1 downto 0);
    constant cond_esp   : in std_logic
) is
begin
    opcode <= op;
    A      <= a_in;
    B      <= b_in;
    wait for PERIODO;

    assert Z = z_esp
        report "ERRO em " & nome & ": Z obtido = " &
            integer'image(to_integer(signed(Z))) &
            ", esperado = " &
            integer'image(to_integer(signed(z_esp)))
        severity error;

    assert cond = cond_esp
        report "ERRO em " & nome & ": cond obtido = " &
            std_logic'image(cond) & ", esperado = " &
            std_logic'image(cond_esp)
        severity error;

    report "OK: " & nome severity note;
end procedure;

begin

-----
-- ADD: resultado positivo, negativo e zero.
-----
check("ADD positivo (10 + 5 = 15)",
    ADD_OP, x"0000000A", x"00000005", x"0000000F", '0');

check("ADD negativo (-10 + 4 = -6)",
    ADD_OP, x"FFFFFFF6", x"00000004", x"FFFFFFFA", '0');

check("ADD zero (-8 + 8 = 0)",
    ADD_OP, x"FFFFFFF8", x"00000008", x"00000000", '0');

-----
-- SUB: resultado positivo, negativo e zero.
-----
check("SUB positivo (20 - 5 = 15)",
    SUB_OP, x"00000014", x"00000005", x"0000000F", '0');

check("SUB negativo (5 - 20 = -15)",
    SUB_OP, x"00000005", x"00000014", x"FFFFFFF1", '0');

check("SUB zero (7 - 7 = 0)",
    SUB_OP, x"00000007", x"00000007", x"00000000", '0');

-----
-- Operacoes logicas.
-----
check("AND (0F0F & 00FF = 000F)",

```



```

        AND_OP, x"0F0F0F0F", x"00FF00FF", x"000F000F", '0');

check("OR (0F0F | 00FF = 0FFF)",
      OR_OP,  x"0F0F0F0F", x"00FF00FF", x"0FFF0FFF", '0');

check("XOR (0F0F ^ 00FF = 0FF0)",
      XOR_OP, x"0F0F0F0F", x"00FF00FF", x"0FF00FF0", '0');

-----
-- Deslocamentos (usa os 5 bits menos significativos de B).
-----
check("SLL (1 << 4 = 16)",
      SLL_OP, x"00000001", x"00000004", x"00000010", '0');

check("SRL (0x80000000 >> 4 sem sinal = 0x08000000)",
      SRL_OP, x"80000000", x"00000004", x"08000000", '0');

check("SRA (0x80000000 >> 4 com sinal = 0xF8000000)",
      SRA_OP, x"80000000", x"00000004", x"F8000000", '0');

-----
-- Comparacoes com sinal.
-----
check("SLT verdadeiro (-1 < 1)",
      SLT_OP, x"FFFFFFFF", x"00000001", x"00000001", '1');

check("SLT falso (1 < -1)",
      SLT_OP, x"00000001", x"FFFFFFFF", x"00000000", '0');

check("SGE verdadeiro (5 >= 5)",
      SGE_OP, x"00000005", x"00000005", x"00000001", '1');

check("SGE falso (-1 >= 1)",
      SGE_OP, x"FFFFFFFF", x"00000001", x"00000000", '0');

-----
-- Comparacoes sem sinal.
-----
check("SLTU verdadeiro (1 < 0xFFFFFFFF)",
      SLTU_OP, x"00000001", x"FFFFFFFF", x"00000001", '1');

check("SLTU falso (0xFFFFFFFF < 1)",
      SLTU_OP, x"FFFFFFFF", x"00000001", x"00000000", '0');

check("SGEU verdadeiro (0xFFFFFFFF >= 1)",
      SGEU_OP, x"FFFFFFFF", x"00000001", x"00000001", '1');

check("SGEU falso (1 >= 0xFFFFFFFF)",
      SGEU_OP, x"00000001", x"FFFFFFFF", x"00000000", '0');

-----
-- Igualdade / diferenca.
-----
check("SEQ verdadeiro (7 == 7)",
      SEQ_OP, x"00000007", x"00000007", x"00000001", '1');

check("SEQ falso (7 == 8)",
      SEQ_OP, x"00000007", x"00000008", x"00000000", '0');

check("SNE verdadeiro (7 != 8)",
      SNE_OP, x"00000007", x"00000008", x"00000001", '1');

```

```
    check("SNE falso (7 != 7)",
          SNE_OP, x"00000007", x"00000007", x"00000000", '0');

    report "Todos os testes da ULA foram executados." severity note;

    wait;

end process;

end architecture tb_arch;
```

Listagem 6 — Testbench autoverificável da ULA.

8. Referências

- PATTERSON, D. A.; HENNESSY, J. L. *Organização e Projeto de Computadores: a interface hardware/software (RISC-V)*. Elsevier.
- WATERMAN, A.; ASANOVIĆ, K. (Ed.). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V International.
- IEEE. *IEEE Std 1076 — VHDL Language Reference Manual*; pacote `numeric_std`.
- Doulos. *Numeric_std Conversions* (guia de conversões de tipos em VHDL).